

St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026

Class: BE-IT A/B, Semester: VII

Subject: Secure Application Development Lab

Student Name: Tanmay Bhatkar

Student Roll No: 14

Experiment – 9 : Study and Implement Symmetric and Asymmetric Encryption Algorithm

Aim: To study and Implement symmetric and asymmetric encryption algorithm

- Symmetric Key Cryptography (Data Encryption Standard)
- Asymmetric Key Cryptography (Diffie Hellman)

Objectives: After study of this experiment, the student will be able to

- understand what is cryptography
- understand the concept of symmetric and asymmetric cryptography
- understand where encryption/decryption standards are used

Lab objective mapped: ITL703.6 To apply secure coding for cryptography

Prerequisite: Basic knowledge of symmetric and asymmetric key cryptography.

Requirements: C/C++/JAVA/PYTHON

Pre-Experiment Theory:

(a) Symmetric Key Cryptography

The Data Encryption Standard (DES) is a symmetric key block cipher algorithm used for data encryption and decryption. It was widely used in the past but is now considered outdated and insecure due to its relatively short key length (56 bits). Nonetheless, understanding how DES works can provide insight into the historical development of encryption algorithms.

Key components of the DES algorithm:

Key Generation:

- The original DES uses a 56-bit key, but only 64 bits are used for the key schedule, with the remaining 8 bits being used for error checking and parity.
- The 56-bit key undergoes a permutation and shifting process to produce 16 subkeys, each of 48 bits in length.

Encryption Process:

- DES operates on 64-bit blocks of plaintext and ciphertext.
- The initial step involves permuting the 64-bit plaintext using a fixed permutation table (initial permutation, IP).
- The 64-bit block is then divided into two halves, each 32 bits in size: the left half (L0) and the right half (R0).

- The core of the DES algorithm consists of 16 rounds, where each round uses the corresponding subkey generated during the key schedule.
- In each round, the right half of the data from the previous round (R_{i-1}) is expanded to 48 bits using an expansion permutation (E-bit selection).
- The expanded right half is then XORed with the current subkey (K_i) to introduce key mixing.
- The XOR result is passed through a series of eight substitution boxes (S-boxes) to perform nonlinear transformations.
- After S-box substitution, a permutation (P-box) is applied to the result.
- The output is then XORed with the left half of the data from the previous round (L_{i-1}).
- The left and right halves are swapped, and the next round begins.

Decryption Process:

- The decryption process in DES is the same as encryption, but the subkeys are used in reverse order.

Final Permutation:

- After completing all 16 rounds, the two halves are combined and undergo a final permutation (inverse initial permutation, IP^{-1}) to produce the final 64-bit ciphertext.

(b) Asymmetric Key Cryptography

Diffie Hellman key exchange is a cryptographic protocol that allows two parties that have no prior knowledge of each other to jointly establish a shared secret key over an insecure communications channel. This key can then be used to encrypt subsequent communications using a symmetric key cipher.

Algorithm:

Alice and Bob, two users who wish to establish secure communications. We can assume that Alice and Bob know nothing about each other but are in contact.

1. Communicating in the clear, Alice and Bob agree on two large positive integers, p and g , where p is a prime number and g is a primitive root mod p .
2. Alice randomly chooses another large positive integer, X_A , which is smaller than p . X_A will serve as Alice's private key.
3. Bob similarly chooses his own private key, X_B .
4. Alice computes her public key, Y_A , using the formula $Y_A = (g^{X_A}) \bmod p$.
5. Bob similarly computes his public key, Y_B , using the formula $Y_B = (g^{X_B}) \bmod p$.
6. Alice and Bob exchange public keys over the insecure circuit.
7. Alice computes the shared secret key, k , using the formula $k = (Y_B^{X_A}) \bmod p$.
8. Bob computes the same shared secret key, k , using the formula $k = (Y_A^{X_B}) \bmod p$.
9. Alice and Bob communicate using the symmetric algorithm of their choice and the shared secret key, k , which was never transmitted over the insecure circuit.

Procedure:

Insert following codes with output

- a. Symmetric Key Cryptography (Data Encryption Standard) code:

from typing import List

--- Tables (DES standard) ---

IP = [58, 50, 42, 34, 26, 18, 10, 2,
60, 52, 44, 36, 28, 20, 12, 4,
62, 54, 46, 38, 30, 22, 14, 6,
64, 56, 48, 40, 32, 24, 16, 8,
57, 49, 41, 33, 25, 17, 9, 1,
59, 51, 43, 35, 27, 19, 11, 3,
61, 53, 45, 37, 29, 21, 13, 5,
63, 55, 47, 39, 31, 23, 15, 7]

IP_INV = [40, 8, 48, 16, 56, 24, 64, 32,
39, 7, 47, 15, 55, 23, 63, 31,
38, 6, 46, 14, 54, 22, 62, 30,
37, 5, 45, 13, 53, 21, 61, 29,
36, 4, 44, 12, 52, 20, 60, 28,
35, 3, 43, 11, 51, 19, 59, 27,
34, 2, 42, 10, 50, 18, 58, 26,
33, 1, 41, 9, 49, 17, 57, 25]

E = [32, 1, 2, 3, 4, 5,
4, 5, 6, 7, 8, 9,
8, 9, 10, 11, 12, 13,
12, 13, 14, 15, 16, 17,
16, 17, 18, 19, 20, 21,
20, 21, 22, 23, 24, 25,
24, 25, 26, 27, 28, 29,
28, 29, 30, 31, 32, 1]

P = [16, 7, 20, 21, 29, 12, 28, 17,
1, 15, 23, 26, 5, 18, 31, 10,
2, 8, 24, 14, 32, 27, 3, 9,
19, 13, 30, 6, 22, 11, 4, 25]

8 S-boxes (standard)

S_BOXES = [
[[14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7],
[0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8],
[4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0],
[15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13]],
[[15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10],
[3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5],
[0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15],
[13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9]],
[[10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8],
[13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1],
[13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7],
[1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12]],
[[7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15],

```

[13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9],
[10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4],
[3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14]],
[[2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9],
[14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6],
[4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14],
[11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3]],
[[12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11],
[10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8],
[9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6],
[4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13]],
[[4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1],
[13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6],
[1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2],
[6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12]],
[[13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7],
[1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2],
[7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8],
[2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11]]
]

```

```

PC1 = [57,49,41,33,25,17,9,
1,58,50,42,34,26,18,
10,2,59,51,43,35,27,
19,11,3,60,52,44,36,
63,55,47,39,31,23,15,
7,62,54,46,38,30,22,
14,6,61,53,45,37,29,
21,13,5,28,20,12,4]

```

```

PC2 = [14,17,11,24,1,5,
3,28,15,6,21,10,
23,19,12,4,26,8,
16,7,27,20,13,2,
41,52,31,37,47,55,
30,40,51,45,33,48,
44,49,39,56,34,53,
46,42,50,36,29,32]

```

```

SHIFT_SCHEDULE = [1, 1, 2, 2, 2, 2, 2, 2,
1, 2, 2, 2, 2, 2, 2, 1]

```

```
# --- Utility Functions ---
```

```
def str_to_bitlist(s: str) -> List[int]:
    return [int(bit) for c in s for bit in f"{ord(c):08b}"]
```

```
def bitlist_to_str(bits: List[int]) -> str:
    chars = [chr(int("".join(map(str, bits[i:i+8])), 2)) for i in range(0, len(bits), 8)]
    return "".join(chars)
```

```

def permute(bits: List[int], table: List[int]) -> List[int]:
    return [bits[i - 1] for i in table]

def xor(a: List[int], b: List[int]) -> List[int]:
    return [i ^ j for i, j in zip(a, b)]

def s_box_substitution(bits: List[int]) -> List[int]:
    output = []
    for i in range(8):
        block = bits[i*6:(i+1)*6]
        row = (block[0] << 1) + block[5]
        col = (block[1] << 3) + (block[2] << 2) + (block[3] << 1) + block[4]
        val = S_BOXES[i][row][col]
        output += [int(x) for x in f"{val:04b}"]
    return output

# --- DES Components ---
def generate_subkeys(key: List[int]) -> List[List[int]]:
    key = permute(key, PC1)
    C, D = key[:28], key[28:]
    keys = []
    for shift in SHIFT_SCHEDULE:
        C = C[shift:] + C[:shift]
        D = D[shift:] + D[:shift]
        keys.append(permute(C + D, PC2))
    return keys

def feistel(right: List[int], subkey: List[int]) -> List[int]:
    expanded = permute(right, E)
    temp = xor(expanded, subkey)
    return permute(s_box_substitution(temp), P)

def des_rounds(bits: List[int], subkeys: List[List[int]]) -> List[int]:
    bits = permute(bits, IP)
    L, R = bits[:32], bits[32:]
    for subkey in subkeys:
        temp = R
        R = xor(L, feistel(R, subkey))
        L = temp
    return permute(R + L, IP_INV)

#ADDING PADDING FOR LARGE CHARACTER HANDLING
def pad_text(text):
    pad_len = 8 - (len(text) % 8)
    return text + chr(pad_len) * pad_len

def unpad_text(padded_text):
    pad_len = ord(padded_text[-1])
    return padded_text[:-pad_len]

```

```

# --- DES Encryption and Decryption ---
def encrypt(plaintext: str, key: str) -> str:
    key_bits = str_to_bitlist(key)
    subkeys = generate_subkeys(key_bits)
    cipher_bits = []

    for i in range(0, len(plaintext), 8): # Process in 8-character blocks
        block = plaintext[i:i+8]
        bits = str_to_bitlist(block)
        encrypted_block = des_rounds(bits, subkeys)
        cipher_bits.extend(encrypted_block)
    return ".join(map(str, cipher_bits))

def decrypt(cipher_bits: List[int], key: str) -> str:
    key_bits = str_to_bitlist(key)
    subkeys = generate_subkeys(key_bits)[::-1]
    decrypted_text = ""

    for i in range(0, len(cipher_bits), 64): # Each block is 64 bits
        block = cipher_bits[i:i+64]
        decrypted_bits = des_rounds(block, subkeys)
        decrypted_text += bitlist_to_str(decrypted_bits)

    return decrypted_text

# --- Main Function ---
def main():
    plaintext = input('Enter the text to be encrypted:') # 64-bit block
    plaintext=pad_text(plaintext) #padding
    key = input("Enter 64 bit key:") # 64-bit key
    print("Plaintext:", plaintext)

    cipher_bits = encrypt(plaintext, key)
    print("Encrypted Bits:", cipher_bits)

    decrypted = decrypt([int(b) for b in cipher_bits], key)
    print("Decrypted Text:", unpad_text(decrypted))

if __name__ == "__main__":
    main()
= RESTART: C:/Users/tanma/Desktop/BE/SADL/DESexp9.py
Enter the text to be encrypted:Attack from north of the wall
Enter 64 bit key:LongLiveTheMotherland
Plaintext: Attack from north of the wall
Encrypted Bits: 0001001101100111000110110000001110101010100001
1100001100110010000101111101100010001101001111100011101001110
01001100000110100111000101010001001010010111000001011001110000
11100110011011100000000110010001001111110011110000000110011111
111011100100010111101011
Decrypted Text: Attack from north of the wall

```

DES Code Output

b. Asymmetric Key Cryptography (Diffie Hellman)

Code:

```
from math import gcd
# prime checking function
def isPrime(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
# Check if alpha is a primitive root modulo q
def is_primitive_root(alpha, q):
    required_set = set(range(1, q))
    actual_set = set(pow(alpha, power, q) for power in range(1, q))
    return required_set == actual_set

# Main DH key exchange logic
def DH(alpha, q, XA, XB):
    if not (isPrime(q) and is_primitive_root(alpha, q) and XA < q and XB < q):
        raise ValueError("INVALID VALUES OF q, alpha, XA or XB")

    YA = pow(alpha, XA, q)
    YB = pow(alpha, XB, q)

    KA = pow(YB, XA, q)
    KB = pow(YA, XB, q)

    assert KA == KB, "ERROR!!!: SHARED KEYS ARE NOT EQUAL!!!!!"
    print(f"Public Key of A: {YA}")
    print(f"Public Key of B: {YB}")
    print(f"Shared Private Key: {KA}")

def main():
    print("Diffie-Hellman Key Exchange")
    q = int(input("Enter your value of q: "))
    XA = int(input("Enter your value of XA: "))
    XB = int(input("Enter your value of XB: "))
    alpha = int(input("Enter your value of alpha: "))

    DH(alpha, q, XA, XB)

if __name__ == "__main__":
    main()
```



```
Diffie-Hellman Key Exchange
Enter your value of q: 23
Enter your value of XA: 6
Enter your value of XB: 15
Enter your value of alpha: 5
Public Key of A: 8
Public Key of B: 19
Shared Private Key: 2
```

Diffie Hellmann Output

Post-Experiments Exercise:

Extended Theory: Nil

Post Experimental Exercise:

Questions:

- Compare symmetric and asymmetric encryption algorithms.
- Discuss the security of DES in the modern context, considering the advancements in cryptanalysis techniques and computing power.
- Discuss real-world applications of the Diffie-Hellman algorithm in various cryptographic protocols, such as secure key exchange for SSL/TLS in web browsers or VPN connections.

Conclusion:

- Write what was performed in the experiment.
- Write the significance of the topic studied in the experiment.

References:

<http://cse29-iiith.vlabs.ac.in/>
